

DOCUMENTATION DU PROJET

E-commune Horison

Géoportail et surveillance agricole de la commune de Blitta
2 (Togo)

Commune	Blitta 2 — Agbandi, Langabou, Koffiti, Tcharè-Baou
Préfecture	Blitta, région Centrale
Contenu	Présentation · Comprendre · Installer et lancer · Architecture · Améliorer
Dépôt	github.com/1998Jules/projet_horieon
Édition	Septembre 2026

Sommaire

PRÉSENTATION (Togo)

E-commune Horison — Géoportail de la commune de Blitta 2

En deux phrases, pour tout le monde
Démarrage rapide (développeurs)

Documentation
Technologies
Données

PARTIE 1

1. Comprendre le projet

1. À quoi sert E-commune ?
2. Le géoportail : la carte de la commune
3. Le module agriculture : surveiller ses champs par satellite

4. La cartothèque : les cartes thématiques
5. L'administration
6. Ce que l'application ne fait pas (encore)
7. Lexique

PARTIE 2

2. Installer et lancer le projet

1. Ce qu'il faut installer
2. Récupérer le code
3. Lancer la base de données
4. Préparer Python
5. Configurer le fichier .env
6. Créer la base, charger les données, créer un compte admin

7. Lancer le serveur
8. Activer le module agriculture (Google Earth Engine)
9. Activer les e-mails d'alerte
10. Automatiser les alertes quotidiennes
11. Relancer le projet les jours suivants
12. Installation sous Linux
13. Dépannage

PARTIE 3

3. Architecture technique

1. Vue d'ensemble
2. Arborescence
3. Configuration (settings)
4. Base de données
5. API et routes

6. Système d'alertes agricoles
7. Données et sources
8. Front-end
9. Commandes de gestion
10. Sécurité

PARTIE 4

4. Améliorer le projet

1. Proposer une modification (fork et Pull Request)
2. Recettes pas à pas

3. Limites connues
4. Feuille de route proposée
5. Contribuer sans coder

E-commune Horison — Géoportail de la commune de Blitta 2 (Togo)

 Lire la documentation complète en PDF (43 pages, mise en page) : [docs/Documentation_E-commune.pdf](#)

E-commune est une application web qui affiche sur une carte interactive les infrastructures d'une commune togolaise (écoles, centres de santé, marchés, points d'eau, routes...) et qui aide les agriculteurs à **surveiller leurs champs par satellite** (santé de la végétation, sécheresse, pluie, alertes par e-mail).

La commune couverte est **Blitta 2** (préfecture de Blitta, région Centrale), formée des cantons d'**Agbandi** (chef-lieu), **Langabou**, **Koffiti** et **Tcharè-Baou**.

En deux phrases, pour tout le monde

- **Le géoportail** : c'est comme Google Maps, mais pour la mairie. On y voit, sur une carte, où se trouvent les écoles, les centres de santé, les marchés, les points d'eau et les routes de la commune, et on peut cliquer dessus pour avoir des informations.
- **Le module agriculture** : un agriculteur dessine son champ sur la carte ; l'application consulte les satellites et la météo, calcule un « score de risque » (sécheresse, manque d'eau, excès de pluie...) et envoie une alerte par e-mail quand la situation se dégrade.

Démarrage rapide (développeurs)

Prérequis : Python 3.12+, Docker, Git. Détails pas à pas, y compris pour les débutants : [docs/2-INSTALLER-ET-LANCER.md](#).

```
git clone https://github.com/1998Jules/projet_horieon.git
cd projet_horieon

# 1. Base de données PostGIS (Docker)
docker run -d --name horieon-postgis -e POSTGRES_PASSWORD=1234 -e POSTGRES_DB=Ecommune \
  -p 127.0.0.1:5440:5432 -v horieon-pgdata:/var/lib/postgresql/data postgis/postgis:16-3.5

# 2. Environnement Python
python -m venv .venv
.venv\Scripts\activate           # Windows (Linux/macOS : source .venv/bin/activate)
```

```

pip install -r requirements.txt
# Windows uniquement : installer aussi la roue GDAL (voir le guide d'installation)

# 3. Configuration
copy .env.exemple .env          # Linux/macOS : cp .env.exemple .env
# puis éditer .env : SECRET_KEY, DEBUG=True, DB_PASSWORD=1234, DB_HOST=127.0.0.1,
DB_PORT=5440

# 4. Base, données, compte admin
python manage.py migrate
python manage.py loaddata cartotheque/fixtures/initial_domaines.json
python manage.py import_blitta2  # couches du géoportail à partir de données ouvertes
python manage.py createsuperuser

# 5. Lancer
python manage.py runserver

```

Puis ouvrir :

Page	Adresse
Géoportail (carte de la commune)	http://127.0.0.1:8000/geoportail/
Module agriculture	http://127.0.0.1:8000/agriculture/
Administration	http://127.0.0.1:8000/admin/
API cartotheque	http://127.0.0.1:8000/api/cartotheque/

La racine <http://127.0.0.1:8000/> affiche une erreur 404 : c'est normal, il n'y a pas de page d'accueil. Utilisez les adresses ci-dessus.

Documentation

 **Version PDF prête à lire ou à imprimer :** docs/Documentation_E-commune.pdf (regénérée par `python docs/generer_pdf.py`).

Guide	Pour qui	Contenu
1. Comprendre le projet	Tout le monde	À quoi sert l'application, comment l'utiliser, lexique (NDVI, SPI, SIG...)
2. Installer et lancer	Débutants et développeurs	Installation pas à pas sous Windows et Linux, dépannage
3. Architecture technique	Développeurs	Structure du code, base de données, API, système d'alertes, configuration

Guide	Pour qui	Contenu
4. Améliorer le projet	Contributeurs	Proposer une modification, ajouter une couche, limites connues, feuille de route

Technologies

Django 6 · Django REST Framework · PostgreSQL + PostGIS · GDAL · Leaflet · Turf.js · Google Earth Engine · CHIRPS / CHIRPS-GEFS · Open-Meteo.

Données

Les données de terrain d'origine (relevés de l'équipe du projet) ne sont pas dans ce dépôt. La commande `import_blitta2` reconstruit les couches à partir de sources ouvertes : limites **OCHA COD-AB**, décret de création des communes (**Journal officiel du 08/01/2018**), infrastructures **OpenStreetMap** (© contributeurs OpenStreetMap, licence ODbL) et inventaire **OMS/KEMRI** des formations sanitaires. Voir [docs/3-ARCHITECTURE.md](#).

1. Comprendre le projet

Ce guide s'adresse à **tout le monde** : élus, agents municipaux, agriculteurs, étudiants, curieux.
Aucune connaissance en informatique n'est nécessaire.

Sommaire

1. À quoi sert E-commune ?
2. Le géoportail : la carte de la commune
3. Le module agriculture : surveiller ses champs par satellite
4. La cartotheque : les cartes thématiques
5. L'administration
6. Ce que l'application ne fait pas (encore)
7. Lexique

1. À quoi sert E-commune ?

Une mairie a besoin de **savoir où se trouvent les choses** sur son territoire pour bien décider : où construire une nouvelle école, quel village est loin d'un centre de santé, quelle piste mérite d'être réhabilitée, où manque l'eau potable...

E-commune rassemble ces informations sur **une seule carte**, consultable depuis un navigateur internet (Chrome, Firefox, Edge...), sans rien installer sur l'ordinateur de l'utilisateur.

L'application a trois parties :

Partie	En une phrase	Pour qui
Géoportail	La carte des infrastructures de la commune	Mairie, services techniques, population
Agriculture	La surveillance des champs par satellite, avec des alertes	Agriculteurs, coopératives, services agricoles
Cartotheque	Une bibliothèque de cartes thématiques prêtes à consulter ou imprimer	Mairie, partenaires, grand public

La commune couverte : Blitta 2

Blitta 2 est l'une des trois communes de la **préfecture de Blitta** (région Centrale du Togo). Elle regroupe quatre **cantons** :

Canton	Rôle
Agbandi	chef-lieu de la commune
Langabou	traversé par la route nationale N1
Koffiti	
Tcharè-Baou	

Cette composition est fixée par le décret publié au *Journal officiel de la République togolaise* du 8 janvier 2018.

2. Le géoportail : la carte de la commune

Adresse : `/geoportail/` (par exemple `http://127.0.0.1:8000/geoportail/` sur l'ordinateur où l'application est lancée).

Ce qu'on voit

- **Un fond de carte** au choix : *OpenStreetMap* (plan), *Satellite* (photo aérienne) ou *Hybride* (photo + noms).
- **Les limites** : le contour de la commune et de ses 4 cantons (en orange).
- **Les routes** : la route nationale revêtue en **rouge épais**, les pistes rurales en **pointillés noirs**.
- **Les infrastructures**, représentées par des petites icônes :

Thème	Couches
Infrastructure	Lycées, collèges, jardins d'enfants, marchés
Hydrographie	Points d'eau autonomes (forages, puits), bornes-fontaines, châteaux d'eau
Santé	Formations sanitaires (USP, CMS, hôpitaux)
Sport et loisirs	Stades et terrains de sport
Agriculture	Coopératives, magasins d'intrants

Comment l'utiliser

1. **Se déplacer** : cliquer-glisser sur la carte. **Zoomer** : molette de la souris ou boutons `+` / `-` en bas à droite.
2. **Afficher des couches** : cliquer sur l'icône orange « **couches** » en haut à droite, puis choisir un **thème** (Santé, Infrastructure...). Changer de fond de carte au même endroit.
3. **Obtenir des informations** : cliquer sur une icône ou un tracé. Une bulle s'ouvre avec le nom, le canton, la localité, etc.
4. **Rechercher** : bouton  **Rechercher** en haut, puis taper un nom (par exemple « Agbandi » ou « Lycée »).

5. **Outils SIG** (panneau de gauche, bouton 

-  **Localiser** : centrer la carte sur votre position (si votre appareil le permet) ;
-  **Dessiner** : tracer un point, une ligne ou une zone sur la carte ;
-  **Mesurer** : mesurer une distance ou une surface ;
- **Filtres & Stats** : compter les infrastructures par canton et par thème (combien de lycées dans le canton de Langabou ?...).

D'où viennent les informations ?

- **À l'origine**, l'équipe du projet a relevé les données sur le terrain (nom des marchés, jour de marché, photos...). Ces données sont dans la base de données de l'auteur du projet, **pas dans le code source**.
- **Sur une nouvelle installation**, une commande reconstruit la carte à partir de **sources publiques** : limites officielles des Nations unies (OCHA), **OpenStreetMap** (la « Wikipédia des cartes », alimentée par des bénévoles) et l'inventaire des centres de santé de l'**OMS**.

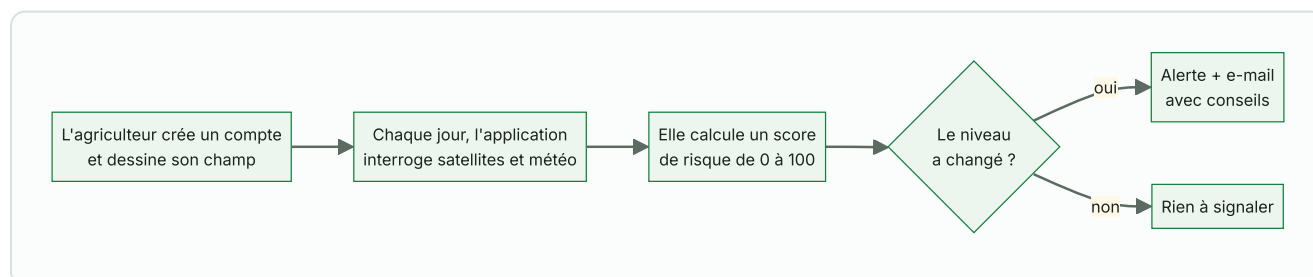
Conséquence : certaines informations peuvent manquer ou être incomplètes. Quand une donnée n'est pas connue, la carte affiche « **Non renseigné** » au lieu d'inventer une valeur. **Chacun peut améliorer OpenStreetMap** (<https://www.openstreetmap.org>) : les ajouts faits là-bas seront repris au prochain import.

3. Le module agriculture : surveiller ses champs par satellite

L'idée

Des satellites (Sentinel-2 de l'Agence spatiale européenne, MODIS de la NASA...) photographient le Togo tous les quelques jours. En analysant les couleurs de ces images, on peut savoir si une plante est **verte et en bonne santé** ou **jaunie et stressée**. En y ajoutant les **relevés de pluie** et les **prévisions météo**, on peut anticiper une sécheresse ou une inondation.

Comment ça marche, étape par étape



1. **L'agriculteur crée un compte et dessine son champ** sur la carte, en indiquant la culture (maïs, igname...) et la date de semis.
2. **Chaque jour** (tâche automatique, par défaut à 6 h), l'application récupère pour chaque champ :
 - la **santé de la végétation** (indices NDVI, EVI, MSAVI) ;

- l'**humidité** de la végétation (NDWI) ;
- la **sécheresse** mesurée et prévue (SPI, VCI, TCI, VHI) ;
- la **pluie** tombée (24 h, 72 h) et **prévue** (16 jours).

3. Elle combine ces indicateurs en **un score de risque de 0 à 100** :

Score	Niveau	Signification
0 – 25	 Normal	Tout va bien
26 – 50	 Vigilance	À surveiller
51 – 75	 Alerte	Agir est recommandé
76 – 100	 Critique	Agir en urgence

4. **Une alerte n'est envoyée que si le niveau change** (aggravation ou amélioration). On ne reçoit donc pas le même e-mail tous les jours.

5. L'e-mail explique **ce qui ne va pas** (stress hydrique, sécheresse, excès de pluie...), **quels indicateurs** l'ont révélé, et donne une **recommandation**.

Le calendrier cultural

L'application contient aussi un **calendrier cultural** : pour chaque culture, les mois de semis, de sarclage et de récolte, et la durée du cycle. Un **simulateur de croissance** indique à quel stade devrait être la culture N jours après le semis.

À savoir

- Cette partie a besoin d'un **accès à Google Earth Engine** (service gratuit pour la recherche et l'usage non commercial) et d'une **adresse e-mail d'envoi**. Sans eux, les calculs satellites et les e-mails ne fonctionnent pas. Voir [le guide d'installation](#).
- L'interface complète pour les agriculteurs (création de compte, dessin des champs, tableau de bord) est une **application séparée** (« frontend »), qui n'est pas dans ce dépôt. Ce dépôt contient le **serveur** (« backend ») qui fait les calculs et fournit les données.

4. La cartothèque : les cartes thématiques

La cartothèque est une **bibliothèque de cartes** déjà mises en page (population, santé, éducation, agriculture...), classées par **domaine**. Chaque carte a un titre, une description, une image, une échelle, un auteur, une source et un statut (brouillon / publiée). Certaines peuvent être payantes.

Les cartes s'ajoutent depuis l'**administration** ; elles sont ensuite disponibles pour l'application frontend via l'adresse `/api/cartotheque/`.

5. L'administration

Adresse : `/admin/` . Réservée aux gestionnaires (identifiant et mot de passe créés lors de l'installation).

On peut y :

- gérer les **utilisateurs** (créer, désactiver, changer un mot de passe) ;
- gérer la **cartothèque** (domaines et cartes) ;
- consulter et corriger les **marchés** (avec photo) ;
- consulter les **champs**, les **alertes** envoyées, les **règles d'alerte** et le **calendrier cultural**.

6. Ce que l'application ne fait pas (encore)

Pour être transparent sur l'état actuel :

- **Pas de page d'accueil** : l'adresse racine du site affiche « Page non trouvée ». Il faut aller directement sur `/geoportail/` .
- **Données de terrain incomplètes** sur une nouvelle installation : jours de marché, dates d'ouverture des écoles, points d'eau, châteaux d'eau, coopératives... ne sont pas dans les sources publiques.
- **La carte du module agriculture** est encore centrée sur Aného et utilise un serveur cartographique (GeoServer) présent seulement sur l'ordinateur de l'auteur.
- **Les listes « Choisir canton » et « Choisir un quartier »** en haut de la carte restent vides (petit défaut du code, et pas de données de quartiers). Utilisez plutôt le filtre **Canton** du panneau *Filtres & Stats*.
- **Pas d'alerte par SMS ou WhatsApp** : seulement par e-mail (le numéro WhatsApp est enregistré mais pas encore utilisé).

La liste détaillée et les pistes d'amélioration sont dans [4-AMELIORER.md](#).

7. Lexique

Terme	Explication simple
SIG	<i>Système d'information géographique</i> : un logiciel qui relie des informations à des positions sur une carte.
Géoportail	Site web qui présente des cartes et des données géographiques.
Couche	Un ensemble d'objets de même nature affichés sur la carte (ex. : la couche « marchés »). On peut l'afficher ou la masquer.
Fond de carte	L'image de base sous les couches (plan, photo satellite...).

Terme	Explication simple
Canton	Subdivision traditionnelle et administrative au Togo, dirigée par un chef canton. Plusieurs cantons forment une commune.
Commune	Collectivité locale dirigée par un conseil municipal et un maire (117 communes au Togo depuis 2019).
USP / CMS	<i>Unité de soins périphérique / Centre médico-social</i> : niveaux de formation sanitaire au Togo.
EPP / CEG	<i>École primaire publique / Collège d'enseignement général</i> .
NDVI	Indice de « verdure » calculé à partir d'images satellites. Proche de 1 : végétation dense et saine. Proche de 0 : sol nu ou plantes en souffrance.
EVI, MSAVI	Variante du NDVI, plus fiables quand la végétation est très dense (EVI) ou clairsemée (MSAVI).
NDWI	Indice d'eau dans la végétation : aide à repérer le manque d'eau des plantes.
SPI	<i>Indice de précipitation standardisé</i> : compare la pluie tombée à la normale des années passées. Négatif = plus sec que d'habitude.
VCI, TCI, VHI	Indices de sécheresse : état de la végétation (VCI), température (TCI) et leur combinaison (VHI).
CHIRPS / CHIRPS-GEFS	Données de pluie observée (CHIRPS) et prévue (CHIRPS-GEFS) produites par l'Université de Californie et l'USGS.
Google Earth Engine	Service de Google qui permet de calculer ces indices sur les images satellites sans les télécharger.
OpenStreetMap (OSM)	Carte du monde libre et collaborative, alimentée par des bénévoles.
API	« Guichet » informatique par lequel une autre application demande des données au serveur.
Frontend / backend	Le frontend est ce que l'utilisateur voit (l'interface) ; le backend est le serveur qui calcule et stocke les données. Ce dépôt est surtout le backend.

2. Installer et lancer le projet

Ce guide part de zéro. Si vous n'avez jamais installé de logiciel de développement, suivez-le dans l'ordre : chaque étape explique **quoi faire** et **comment vérifier que ça a marché**. Les développeurs expérimentés peuvent sauter directement au **démarrage rapide du README**.

Sommaire

1. Ce qu'il faut installer
2. Récupérer le code
3. Lancer la base de données
4. Préparer Python
5. Configurer le fichier .env
6. Créer la base, charger les données, créer un compte admin
7. Lancer le serveur
8. Activer le module agriculture (Google Earth Engine)
9. Activer les e-mails d'alerte
10. Automatiser les alertes quotidiennes
11. Relancer le projet les jours suivants
12. Installation sous Linux
13. Dépannage

1. Ce qu'il faut installer

Logiciel	À quoi il sert	Où le trouver
Python 3.12 ou plus récent	Le langage dans lequel l'application est écrite	https://www.python.org/downloads/
Git	Télécharger le code et suivre les modifications	https://git-scm.com/downloads
Docker Desktop	Faire tourner la base de données sans l'installer « à la main »	https://www.docker.com/products/docker-desktop/
Un éditeur de code (conseillé)	Modifier les fichiers	https://code.visualstudio.com/

⚠ **Python** : pendant l'installation sous Windows, **cochez la case « Add python.exe to PATH »** en bas de la première fenêtre. Django 6 exige **Python 3.12 minimum** : les versions 3.10 et 3.11 ne fonctionnent pas.

Ouvrir un terminal

Toutes les commandes de ce guide se tapent dans un **terminal** :

- **Windows** : touche Windows, tapez `PowerShell` , puis Entrée.
- **Linux / macOS** : application « Terminal ».

Vérifier les installations

Tapez ces commandes une par une. Chacune doit afficher un numéro de version :

```
python --version      # attendu : Python 3.12.x, 3.13.x ou 3.14.x
git --version
docker --version
```

Si une commande répond « n'est pas reconnu », le logiciel n'est pas installé ou n'est pas dans le PATH : réinstallez-le, puis **fermez et rouvrez** le terminal.

Plusieurs versions de Python ? Sous Windows, `py -0` les liste. Utilisez alors `py -3.14` (ou la version voulue) à la place de `python` à l'étape 4.

2. Récupérer le code

Choisissez un dossier de travail (par exemple `Documents\Projets`), puis :

```
cd $HOME\Documents
git clone https://github.com/1998Jules/projet_horieon.git
cd projet_horieon
```

✓ **Vérification** : `dir` (Windows) ou `ls` (Linux) affiche notamment `manage.py` , `requirements.txt` , et les dossiers `geoportail` , `agriculture` ...

Toutes les commandes suivantes se tapent depuis ce dossier `projet_horieon` .

3. Lancer la base de données

L'application stocke ses données dans **PostgreSQL** avec l'extension géographique **PostGIS**. Le plus simple est de la lancer avec Docker.

1. Démarrez **Docker Desktop** (l'icône de la baleine doit être stable, pas en train de clignoter).
2. Tapez (sur **une seule ligne**) :

```
docker run -d --name horieon-postgis -e POSTGRES_PASSWORD=1234 -e POSTGRES_DB=Ecommune -p 127.0.0.1:5440:5432 -v horieon-pgdata:/var/lib/postgresql/data --restart unless-stopped postgis/postgis:16-3.5
```

Ce que fait cette commande :

Élément	Signification
<code>--name horieon-postgis</code>	Nom du conteneur, pour le retrouver
<code>POSTGRES_PASSWORD=1234</code>	Mot de passe de l'utilisateur <code>postgres</code> (changez-le si l'ordinateur est partagé ou exposé sur un réseau)
<code>POSTGRES_DB=Ecommune</code>	Crée directement la base <code>Ecommune</code>
<code>-p 127.0.0.1:5440:5432</code>	La base est accessible sur le port 5440 de votre ordinateur, uniquement en local. On évite le port 5432, souvent déjà pris par un PostgreSQL installé
<code>-v horieon-pgdata:...</code>	Les données sont conservées même si le conteneur est supprimé
<code>--restart unless-stopped</code>	La base redémarre toute seule avec Docker

✓ Vérification :

```
docker exec horieon-postgis psql -U postgres -d Ecommune -c "select postgis_version();" 
```

doit afficher une ligne commençant par `3.5` .

▼ Alternative sans Docker (PostgreSQL installé sur Windows)

1. Installez PostgreSQL depuis <https://www.postgresql.org/download/windows/>.
2. Lancez **Application Stack Builder** (menu Démarrer > PostgreSQL), puis choisissez *Spatial Extensions* > *PostGIS*.
3. Créez la base : `psql -U postgres -c "CREATE DATABASE \"Ecommune\";"` .
4. Dans `.env` (étape 5), mettez `DB_PORT=5432` et votre mot de passe `postgres` .

4. Préparer Python

4.1 Créer un environnement virtuel

Un **environnement virtuel** (`.venv`) est un dossier qui contient les bibliothèques du projet, sans rien mélanger avec le reste de l'ordinateur.

```
python -m venv .venv
.venv\Scripts\activate
```

✓ **Vérification** : le début de la ligne du terminal affiche `(.venv)` .

PowerShell refuse d'activer (« l'exécution de scripts est désactivée ») ? Tapez une fois `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` , répondez `0` , puis relancez `.venv\Scripts\activate` .

4.2 Installer les bibliothèques

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Cela prend quelques minutes (Django, pandas, rasterio, Earth Engine...).

4.3 Installer GDAL (Windows uniquement)

Django a besoin de **GDAL** et **GEOS**, deux bibliothèques de calcul géographique. Sous Windows, on les installe avec une « roue » précompilée :

1. Allez sur <https://github.com/cgohlke/geospatial-wheels/releases> (dernière version).
2. Dans la liste des fichiers (*Assets*), repérez celui qui commence par `gdal-` et qui correspond à **votre version de Python** : `cp312` = Python 3.12, `cp313` = 3.13, `cp314` = 3.14, et se termine par `win_amd64.whl` . Exemple : `gdal-3.13.3-cp314-cp314-win_amd64.whl` .
3. Installez-le directement depuis son lien :

```
pip install https://github.com/cgohlke/geospatial-wheels/releases/download/v2026.8.20/gdal-3.13.3-cp314-cp314-win_amd64.whl
```

(adaptez le lien au fichier choisi).

✓ **Vérification** :

```
python -c "from osgeo import gdal; print(gdal.__version__)"
```

Le fichier `horison/settings.py` trouve ensuite tout seul `gdal.dll` , `geos_c.dll` et les données PROJ dans le dossier `.venv` .

5. Configurer le fichier .env

Le fichier `.env` contient les réglages propres à votre ordinateur et **les secrets** (mots de passe). Il n'est **jamais** envoyé sur GitHub.

```
copy .env.exemple .env          # Linux/macOS : cp .env.exemple .env
```

Générez une clé secrète :

```
python -c "from django.core.management.utils import get_random_secret_key as g; print(g())"
```

Puis ouvrez `.env` avec un éditeur (`notepad .env`) et remplissez :

```
SECRET_KEY=la-cle-generée-ci-dessus
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

DB_NAME=Ecommune
DB_USER=postgres
DB_PASSWORD=1234
DB_HOST=127.0.0.1
DB_PORT=5440
```

Variable	Rôle	Valeur par défaut si absente
SECRET_KEY	Clé de chiffrement des sessions. Unique et secrète en production.	clé de développement non sûre
DEBUG	<code>True</code> affiche les erreurs détaillées et sert les fichiers statiques. <code>False</code> en production.	<code>False</code>
ALLOWED_HOSTS	Noms de domaine autorisés, séparés par des virgules	<code>localhost,127.0.0.1</code>
DB_NAME / DB_USER / DB_PASSWORD / DB_HOST / DB_PORT	Connexion à PostgreSQL	<code>Ecommune / postgres / 1234 / localhost / 5432</code>
EMAIL_HOST , EMAIL_PORT , EMAIL_USE_TLS	Serveur d'envoi des e-mails	<code>smtp.gmail.com , 587 , True</code>
EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	Compte et mot de passe d'envoi (voir étape 9)	vide (pas d'envoi)
DEFAULT_FROM_EMAIL	Expéditeur affiché	<code>= EMAIL_HOST_USER</code>
ALERT_RECIPIENT_EMAIL	Adresse de contrôle qui reçoit les alertes	vide

Une variable d'environnement définie dans le terminal **est prioritaire** sur la valeur du fichier `.env`.

6. Créer la base, charger les données, créer un compte admin

```
python manage.py migrate
```

Crée les tables de l'application.  Se termine par une série de `OK`.

```
python manage.py loaddata cartotheque/fixtures/initial_domaines.json
```

Charge les 14 domaines de la cartotheque.  `Installed 14 object(s)`.

```
python manage.py import_blitta2
```

Télécharge les données ouvertes (limites officielles, OpenStreetMap, inventaire OMS) et **remplit les couches du géoportail**. Il faut une connexion internet ; comptez une à deux minutes.  Se termine par `Import terminé` et le nombre d'objets par table.

 Cette commande **efface et remplace** le contenu des couches du géoportail. Ne la lancez pas sur une base qui contient les relevés de terrain d'origine.

```
python manage.py createsuperuser
```

Crée votre compte administrateur : choisissez un identifiant, une adresse e-mail et un mot de passe (rien ne s'affiche pendant la saisie du mot de passe, c'est normal).

7. Lancer le serveur

```
python manage.py runserver
```

 Le terminal affiche `Starting development server at http://127.0.0.1:8000/`. **Laissez ce terminal ouvert** tant que vous utilisez l'application (`Ctrl + C` pour l'arrêter).

Ouvrez votre navigateur :

Page	Adresse
Géoportail	http://127.0.0.1:8000/geoportail/
Module agriculture	http://127.0.0.1:8000/agriculture/
Administration	http://127.0.0.1:8000/admin/
API cartotheque	http://127.0.0.1:8000/api/cartotheque/

Le port 8000 est déjà utilisé ? Lancez sur un autre port : `python manage.py runserver 127.0.0.1:8002`, puis utilisez `http://127.0.0.1:8002/...`

8. Activer le module agriculture (Google Earth Engine)

Les calculs satellites (NDVI, sécheresse...) passent par **Google Earth Engine**, gratuit pour un usage non commercial (recherche, ONG, collectivités).

1. Avec un compte Google, créez un projet sur <https://console.cloud.google.com/> (menu *Sélectionner un projet > Nouveau projet*).
2. Enregistrez ce projet pour Earth Engine sur <https://code.earthengine.google.com/register> : choisissez *usage non commercial* puis le projet créé.
3. Dans la console Cloud : *API et services > Bibliothèque*, puis activez **Google Earth Engine API**.
4. *IAM et administration > Comptes de service > Créer un compte de service*. Donnez-lui le rôle **Earth Engine Resource Writer** (ou *Viewer* pour la lecture seule) et **Service Usage Consumer**.
5. Ouvrez ce compte de service, onglet *Clés*, puis *Ajouter une clé > JSON*. Un fichier `.json` est téléchargé.
6. **Copiez ce fichier à la racine du projet** (à côté de `manage.py`) sous le nom attendu par le code :

```
ee-koutoumbogajules-c99000ca569e.json
```

Ce nom est codé en dur dans `agriculture/gee_utils.py` (constante `SERVICE_ACCOUNT_KEY_FILE`). Le fichier est déjà exclu de Git par le `.gitignore`. Le serveur doit être lancé **depuis la racine du projet**, car le chemin est relatif.

⚠ Ce fichier donne accès à votre compte Google Cloud : **ne le partagez jamais** et ne le committez pas.

9. Activer les e-mails d'alerte

Exemple avec Gmail :

1. Activez la **validation en deux étapes** sur le compte Google d'envoi.
2. Créez un **mot de passe d'application** sur <https://myaccount.google.com/apppasswords> (nom : « E-commune »). Google affiche 16 caractères.
3. Complétez `.env` :

```
EMAIL_HOST_USER=adresse.envoi@gmail.com
EMAIL_HOST_PASSWORD=les16caracteres
```

```
DEFAULT_FROM_EMAIL=adresse.envoi@gmail.com
ALERT_RECIPIENT_EMAIL=adresse.de.controle@exemple.com
```

4. Redémarrez le serveur.

✓ **Test :** `python manage.py evaluate_field_alerts --dry-run` calcule les alertes sans rien enregistrer ni envoyer.

Pour tester sans vrai envoi, ajoutez

`EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend` dans `.env` : les e-mails s'affichent alors dans le terminal.

10. Automatiser les alertes quotidiennes

La commande qui fait tout (collecte des indicateurs, score, alertes, e-mails) :

```
python manage.py evaluate_field_alerts          # tous les champs
python manage.py evaluate_field_alerts --champ-id 5  # un seul champ
python manage.py evaluate_field_alerts --dry-run    # simulation
python manage.py evaluate_field_alerts --no-collect  # sans réinterroger les satellites
python manage.py evaluate_field_alerts --no-email   # sans envoyer d'e-mail
```

Windows : `install_alerts_task.ps1` crée une tâche planifiée qui lance `run_alerts.bat` chaque jour à 6 h. ⚠ Ces deux fichiers contiennent des chemins propres à l'ordinateur de l'auteur

(`D:\Horison\...`) : **adaptez d'abord** `$BatchPath` dans `install_alerts_task.ps1`, ainsi que les lignes `cd /d`, `call ...\activate.bat` et le chemin du journal dans `run_alerts.bat`. Lancez ensuite PowerShell **en administrateur** :

```
.\install_alerts_task.ps1          # tous les jours à 06:00
.\install_alerts_task.ps1 -Time "08:30"  # autre heure
.\install_alerts_task.ps1 -Interval 6    # toutes les 6 heures
.\install_alerts_task.ps1 -Test         # lancer immédiatement
.\install_alerts_task.ps1 -Uninstall     # supprimer la tâche
```

Linux : ajoutez une ligne dans `crontab -e` :

```
0 6 * * * cd /chemin/vers/projet_horieon && .venv/bin/python manage.py evaluate_field_alerts
>> alerts.log 2>&1
```

11. Relancer le projet les jours suivants

```
cd $HOME\Documents\projet_horieon
.venv\Scripts\activate
python manage.py runserver
```

Docker Desktop doit être démarré (la base redémarre toute seule grâce à `--restart unless-stopped`).
Si ce n'est pas le cas : `docker start horieon-postgis` .

Après avoir récupéré des modifications (`git pull`), pensez à :

```
pip install -r requirements.txt
python manage.py migrate
```

12. Installation sous Linux

Testé sur Debian/Ubuntu :

```
sudo apt update
sudo apt install -y python3 python3-venv python3-dev gdal-bin libgdal-dev git docker.io
sudo usermod -aG docker $USER      # puis se déconnecter / reconnecter

git clone https://github.com/1998Jules/projet_horieon.git && cd projet_horieon
docker run -d --name horieon-postgis -e POSTGRES_PASSWORD=1234 -e POSTGRES_DB=Ecommune \
  -p 127.0.0.1:5440:5432 -v horieon-pgdata:/var/lib/postgresql/data --restart unless-stopped
postgis/postgis:16-3.5

python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.exemple .env && nano .env
python manage.py migrate
python manage.py loaddata cartotheque/fixtures/initial_domaines.json
python manage.py import_blitta2
python manage.py createsuperuser
python manage.py runserver
```

Sous Linux, **pas besoin de la roue GDAL** : Django utilise la bibliothèque système installée par `apt` .

Mise en production (aperçu)

Le script `build.sh` (installation, `collectstatic` , `migrate`) est prévu pour un hébergeur comme Render. En production :

- `DEBUG=False` , une vraie `SECRET_KEY` , `ALLOWED_HOSTS=mon-domaine.tg` ;

- servir l'application avec **gunicorn** : `gunicorn horison.wsgi:application --bind 0.0.0.0:8000` ;
- servir `staticfiles/` et `media/` par le serveur web (Nginx...) ;
- une base PostGIS gérée, avec des sauvegardes (`pg_dump`).

13. Dépannage

Symptôme	Cause probable	Solution
Page not found (404) sur <code>http://127.0.0.1:8000/</code>	Pas de page d'accueil	Aller sur <code>/geoportail/</code>
Error: That port is already in use	Un autre programme utilise le port 8000	<code>python manage.py runserver 127.0.0.1:8002</code>
<code>connection refused / could not connect to server</code>	La base ne tourne pas	Démarrer Docker Desktop, puis <code>docker start horieon-postgis</code>
<code>password authentication failed for user</code>	Mauvais mot de passe ou mauvais port dans <code>.env</code>	Vérifier <code>DB_PASSWORD</code> et <code>DB_PORT</code> (5440 pour le conteneur)
<code>UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9</code> au démarrage	Échec de connexion à un PostgreSQL configuré en français : le message d'erreur accentué fait planter <code>psycpg2</code>	C'est en réalité l'erreur ci-dessus : vérifier <code>DB_*</code> dans <code>.env</code>
<code>Could not find the GDAL library / OSError: [WinError 126]</code>	Roue GDAL absente ou pour une autre version de Python	Refaire l'étape 4.3 avec le bon <code>cpXXX</code>
<code>Django requires Python 3.12 or later</code>	Python trop ancien	Installer Python 3.12+ et recréer le <code>.env</code>
Invalid requirement: <code>'D\x00j\x00a...'</code>	Ancien <code>requirements.txt</code> encodé en UTF-16	Récupérer la version actuelle du dépôt (<code>git pull</code>)
<code>relation "bl2" does not exist</code> (erreur 500 sur les couches)	Tables du géoportail absentes	<code>python manage.py import_blitta2</code>
Erreur 500 sur <code>/agriculture/api/regions/</code> , <code>/prefectures/</code> ou <code>/communes/</code>	Tables <code>region</code> , <code>couche_prefecture_utm</code> , <code>communes_togo_utm</code> absentes (importées à la main par l'auteur)	Voir 4-AMELIORER.md
Fond de carte remplacé par des cases « Access blocked »	Ancienne version des fichiers JavaScript (tuiles OSM sans Referer)	Récupérer la version actuelle et vider le cache du navigateur (<code>Ctrl + Maj + R</code>)
La carte est vide mais sans erreur	Couches non activées	Icône orange en haut à droite, puis choisir un thème
ERREUR: Le fichier clé <code>'ee-...json'</code> est introuvable	Clé Earth Engine absente	Étape 8 ; lancer le serveur depuis la racine du projet

Symptôme	Cause probable	Solution
SMTPAuthenticationError	Mot de passe Gmail normal au lieu d'un mot de passe d'application	Étape 9
.venv\Scripts\activate refusé	Politique d'exécution PowerShell	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

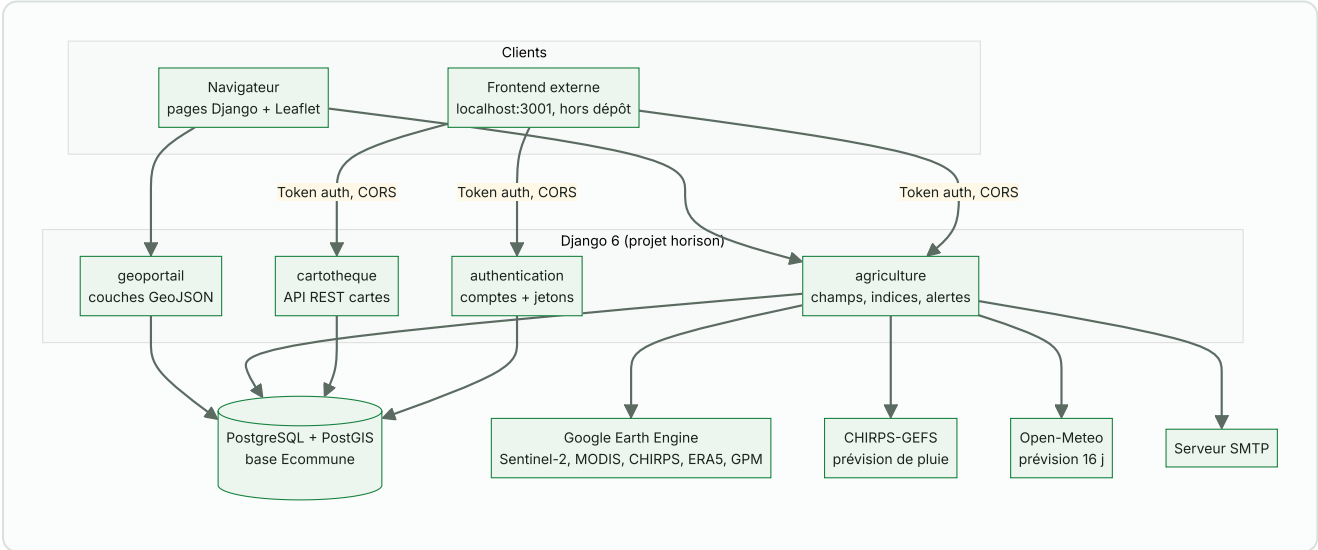
3. Architecture technique

Pour les **développeurs**. Ce document décrit l'organisation du code, la base de données, les API, le système d'alertes et la configuration. Pré-requis : connaître les bases de Django.

Sommaire

- 1. Vue d'ensemble
- 2. Arborescence
- 3. Configuration (settings)
- 4. Base de données
- 5. API et routes
- 6. Système d'alertes agricoles
- 7. Données et sources
- 8. Front-end
- 9. Commandes de gestion
- 10. Sécurité

1. Vue d'ensemble



Couche	Technologie
Langage / framework	Python 3.12+, Django 6.0

Couche	Technologie
API REST	Django REST Framework 3.x, <code>rest_framework.authtoken</code> , <code>django-filter</code>
SIG serveur	<code>django.contrib.gis</code> (GeoDjango), GDAL/GEOS/PROJ
Base de données	PostgreSQL 16 + PostGIS 3.5
Cartographie navigateur	Leaflet 1.9, Leaflet.draw, Turf.js 6, proj4js
Télédétection	Google Earth Engine (<code>earthengine-api</code>), <code>rasterio</code> , <code>scipy</code>
Météo	CHIRPS / CHIRPS-GEFS (UCSB), Open-Meteo
CORS	<code>django-cors-headers</code>
Déploiement	<code>build.sh</code> (Render), <code>gunicorn</code>

2. Arborescence

```

projet_horieon/
├─ manage.py                # point d'entrée Django
(DJANGO_SETTINGS_MODULE=horison.settings)
├─ requirements.txt         # dépendances pip (UTF-8)
├─ .env.exemple             # modèle de configuration (copier en .env)
├─ build.sh                 # build de déploiement : pip install, collectstatic, migrate
├─ run_alerts.bat           # lanceur Windows de evaluate_field_alerts (chemins à
adapter)
├─ install_alerts_task.ps1  # crée la tâche planifiée Windows
├─ commune.geojson          # ancien fichier de test (zone d'Aného, EPSG:32631),
inutilisé
├─ model.py                 # brouillon de modèles, inutilisé
├─ urls.py, wsgi.py, asgi.py # doublons historiques de horison/, inutilisés
|
├─ horison/                 # le « projet » Django
|   ├─ settings.py          # configuration (lit .env) + détection GDAL sous Windows
|   ├─ urls.py              # routes racine
|   └─ wsgi.py / asgi.py
|
├─ geoportail/              # carte communale
|   ├─ models.py            # 14 couches (modèles NON gérés, managed=False)
|   ├─ views.py             # endpoints GeoJSON, recherche, toutes couches
|   └─ urls.py
|   ├─ admin.py             # admin des marchés (aperçu photo)
|   └─ management/commands/import_blitaa2.py # reconstruction des couches (données
ouvertes)
|   └─ templates/index.html # page du géoportail

```



```

|   └─ static/                                # js/script.js (carte), css/, icone/*.png, images/
|
|─ agriculture/                               # suivi des champs et alertes
|   └─ models.py                             # Champ, CropCalendar, FieldIndicatorSnapshot, AlertRule,
|       |                                     # FarmerAlert, RiskAssessment, AlertDelivery (+ 3 couches non
gérées)
|   └─ views.py                             # ~25 endpoints JSON
|   └─ gee_utils.py                         # tous les calculs Earth Engine (NDVI, SPI, VHI, ERA5, GPM...)
|   └─ chirps_gefs.py                       # prévision de pluie CHIRPS-GEFS (rasterio)
|   └─ weather_forecast.py                  # prévision Open-Meteo (sécheresse / inondation à venir)
|   └─ indicator_collector.py               # 1. collecte → FieldIndicatorSnapshot
|   └─ risk_engine.py                       # 2. score de risque 0-100
|   └─ alert_services.py                    # 3. création des alertes si le niveau change
|   └─ notification_services.py             # 4. e-mails HTML
|   └─ management/commands/evaluate_field_alerts.py # orchestre 1 → 4
|   └─ migrations/0008_default_alert_rules.py    # règles d'alerte par défaut
|   └─ templates/agriculture/index.html
|   └─ static/js/main.js
|
|─ authentication/                           # inscription / connexion par jeton
|   └─ models.py                             # UserProfile (secteur, WhatsApp, alertes oui/non)
|   └─ serializers.py, views.py, urls.py
|
|─ cartotheque/                             # bibliothèque de cartes thématiques
|   └─ models.py                             # DomaineCarte, CarteTheematique
|   └─ views.py                             # ViewSets DRF
|   └─ api_urls.py                          # router DRF
|   └─ fixtures/initial_domaines.json
|
|─ media/                                    # fichiers envoyés (photos de marchés, images de cartes)
└─ docs/                                    # cette documentation

```

3. Configuration (settings)

Fichier : `horison/settings.py` .

Chargement

1. `BASE_DIR` = racine du dépôt.
2. `load_dotenv(BASE_DIR / '.env')` : les variables du fichier `.env` sont chargées **sans écraser** celles déjà définies dans l'environnement.
3. Fonctions utilitaires `env_bool()` et `env_list()` .

GDAL / GEOS / PROJ

- **Windows** (`os.name == 'nt'`) : le code cherche la roue Python GDAL dans `sys.prefix/Lib/site-packages/osgeo` (le venv courant) et, si les fichiers existent, définit `GDAL_LIBRARY_PATH` , `GEOS_LIBRARY_PATH` , `PROJ_LIB` , `GDAL_DATA` et ajoute `osgeo` au `PATH` .
- **Linux** : rien n'est forcé ; GeoDjango trouve `libgdal` / `libgeos_c` installés par le système.
- `PROJ_NETWORK=OFF` : PROJ n'essaie pas de télécharger de grilles.

Paramètres lus depuis l'environnement

Voir le tableau complet dans [2-INSTALLER-ET-LANCER.md > étape 5](#).

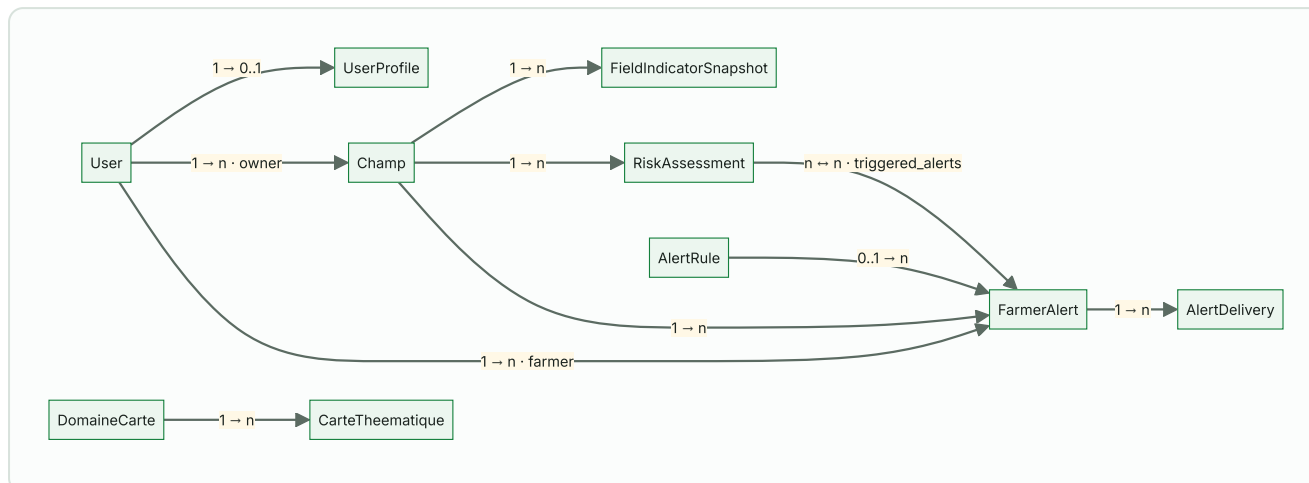
Autres réglages notables

Réglage	Valeur	Remarque
<code>REST_FRAMEWORK</code>	<code>TokenAuthentication</code> , <code>AllowAny</code>	Chaque vue restreint elle-même si besoin
<code>CORS_ALLOW_ALL_ORIGINS</code>	<code>True</code>	Pratique en développement, à restreindre en production (<code>CORS_ALLOWED_ORIGINS</code>)
<code>CORS_ALLOW_CREDENTIALS</code>	<code>True</code>	
<code>X_FRAME_OPTIONS</code>	<code>ALLOW-FROM</code> <code>http://localhost:3001</code>	Directive obsolète, ignorée par les navigateurs récents
<code>STATIC_ROOT</code> / <code>MEDIA_ROOT</code>	<code>staticfiles/</code> / <code>media/</code>	
<code>LANGUAGE_CODE</code> / <code>TIME_ZONE</code>	<code>en-us</code> / <code>UTC</code>	
<code>NDVI_ALERT_THRESHOLD</code>	<code>0.35</code>	Seuil historique de stress hydrique
<code>DEFAULT_AUTO_FIELD</code>	<code>BigAutoField</code>	

4. Base de données

Base PostgreSQL `Ecommune` avec l'extension **PostGIS**. Toutes les géométries sont en **WGS 84 (EPSG:4326)**.

4.1 Tables gérées par les migrations



Lecture : « Champ 1 → n FieldIndicatorSnapshot » = un champ possède plusieurs mesures d'indicateurs.

Modèle	Table	Rôle
authentication.UserProfile	authentication_userprofile	Secteur d'activité, WhatsApp, accepte les alertes
agriculture.Champ	agriculture_champ	Parcelle : nom, propriétaire, culture, date de semis, geom (GeometryField 4326), owner
agriculture.CropCalendar	agriculture_crop_calendar	Culture, variété, type (céréale, légume, tubercule, légumineuse), durée du cycle, mois de semis / sarclage / récolte
agriculture.FieldIndicatorSnapshot	agriculture_field_indicatorsnapshot	Une valeur d'indicateur pour un champ à une date (+ référence, unité, source, métadonnées JSON)
agriculture.AlertRule	agriculture_alert_rule	Règle : indicateur, opérateur (LT , LTE , GT , GTE , EQ , CHANGE_PCT), seuil, sévérité, gabarits de message, délai anti-répétition
agriculture.FarmerAlert	agriculture_farmer_alert	Alerte émise : type, niveau, score, indicateurs contributeurs, message, recommandation, statut (OPEN , ACKNOWLEDGED , RESOLVED , EXPIRED)
agriculture.AlertDelivery	agriculture_alert_delivery	Envoi d'une alerte : canal (EMAIL , WHATSAPP), destinataire, statut (PENDING , SENT , FAILED , SKIPPED), tentatives
cartotheque.DomaineCarte	cartotheque_domaine_carte	Domaine thématique (icône Lucide, couleur, ordre)
cartotheque.CarteTheematique	cartotheque_carte_theematique	Carte : image, vignette, GeoJSON, centre / zoom, fond, statut (brouillon , publie ...), échelle, format, prix

Plus les tables Django standard (auth_*, django_*, authtoken_token).

4.2 Tables NON gérées (`managed = False`)

Ces modèles décrivent des tables **créées hors de Django** (import de shapefiles avec QGIS ou `shp2pgsql`). `migrate` ne les crée pas.

Modèle	Table	Géométrie	Champs principaux	Créée par <code>import_blitta2</code>
<code>geoportail.Commune</code>	<code>bl2</code>	MultiPolygon	commune, code_commu, region, prefecture	✓
<code>geoportail.Cantons</code>	<code>cant_bli2</code>	MultiPolygon	gid (PK), canton, code_canto	✓
<code>geoportail.routes</code>	<code>routeb2</code>	MultiLineString	route_type, route_clas, route_reco, route_nom	✓
<code>geoportail.marche</code>	<code>marches</code>	Point	marche_nom, jour, photo	✓
<code>geoportail.lycee / college / jardin</code>	<code>lyc2 / collegeb2 / jardb2</code>	Point	etablissem, etab_adr, ouverture, etabliss_1 (statut), inspection, ministere	✓
<code>geoportail.pea</code>	<code>pea</code>	Point	forage_nom, forage_typ, batiment_n	✓
<code>geoportail.bornefontaine</code>	<code>bornefontaines</code>	Point	borne_font	✓
<code>geoportail.chateau</code>	<code>chatea</code>	Point	chateau_no, organisme	✓
<code>geoportail.hopital</code>	<code>formation_s</code>	Point	nom_fs, secteur, services_p	✓
<code>geoportail.terrain</code>	<code>stade_terrainbl2</code>	MultiPolygon	terrain, terrain_sp	✓
<code>geoportail.cooperative</code>	<code>cooperativeb2</code>	Point	cooperativ, cooperat_1	✓
<code>geoportail.magasinbl2</code>	<code>magazin_intrantbl2</code>	Point	etab_nom, organisme	✓
<code>agriculture.Region</code>	<code>region</code>	MultiPolygon	region	✗
<code>agriculture.Prefecture</code>	<code>couche_prefecture_utm</code>	MultiPolygon	prefecture	✗
<code>agriculture.Commune</code>	<code>communes_togo_utm</code>	MultiPolygon	commune, prefecture	✗

Toutes les couches ponctuelles du géoportail ont en plus `canton_nom` et, sauf exception, `nom_locali` (localité).

Les valeurs de `routeb2.route_type` sont **utilisées par le style** de la carte : seules `Route nationale revêtue` et `Piste rurale` sont dessinées (comparaison insensible à la casse et aux accents, fonction `styleRoute` dans `geoportail/static/js/script.js`).

5. API et routes

Routes racine (`horison/urls.py`) : `admin/` , `auth/` , `geoportail/` , `agriculture/` , `api/cartotheque/` , plus le service des fichiers `media/` en mode `DEBUG` .

5.1 Authentification — `/auth/`

Authentification par **jeton** : envoyer l'en-tête `Authorization: Token <clé>` (les vues `agriculture` acceptent aussi `Bearer`).

Méthode	Route	Accès	Corps / réponse
POST	<code>/auth/reg</code> <code>ister/</code>	public	<code>username</code> , <code>email</code> , <code>first_name</code> , <code>last_name</code> , <code>password</code> , <code>password_confirm</code> , <code>secteur_activite?</code> , <code>whatsapp?</code> , <code>recevoir_alertes?</code> → <code>{token, user}</code>
POST	<code>/auth/log</code> <code>in/</code>	public	<code>username</code> (ou <code>identifiant</code>), <code>password</code> → <code>{token, user}</code>
GET	<code>/auth/me</code> <code>/</code>	connecté	profil de l'utilisateur
POST	<code>/auth/log</code> <code>out/</code>	connecté	supprime le jeton

5.2 Géoportail — `/geoportail/`

Méthode	Route	Réponse
GET	<code>/geoportail/</code>	page HTML de la carte
GET	<code>/geoportail/geojson/commune/</code>	FeatureCollection <code>bl2</code>
GET	<code>/geoportail/geojson/cantons/</code>	FeatureCollection <code>cant_bli2</code>
GET	<code>/geoportail/geojson/routes/</code>	<code>routeb2</code>
GET	<code>/geoportail/geojson/lycee/</code> , <code>college/</code> , <code>jardin/</code>	établissements scolaires
GET	<code>/geoportail/geojson/Point_eau/</code> , <code>bornefontaine/</code> , <code>chateau/</code>	eau
GET	<code>/geoportail/geojson/Marches/</code>	marchés
GET	<code>/geoportail/geojson/hopitale/</code>	formations sanitaires
GET	<code>/geoportail/api/layers/</code>	toutes les couches en un seul appel

Méthode	Route	Réponse
GET	/geoportail/api/search/?q=<texte>&layer=<couche>	recherche plein texte (15 résultats max par couche)

Les vues `terrain_geojson`, `cooperative_geojson` et `magazin_geojson` existent dans `views.py` mais **ne sont pas routées** dans `urls.py`.

5.3 Agriculture — /agriculture/

Sauf mention contraire, les POST attendent un corps JSON.  = jeton requis ; l'utilisateur ne voit que **ses** champs (ou tous s'il est `staff`).

Méthode	Route		Rôle
GET	/agriculture/		page HTML
GET	api/regions/ , api/prefectures/ , api/communes/		limites administratives (tables non gérées)
GET	api/ndvi-tiles/		URL de tuiles NDVI Earth Engine
POST	api/ndvi-timeseries/		série NDVI mensuelle d'une zone (<code>zone_type</code> , <code>zone_id</code> , <code>years</code>)
POST	api/drought-indices/		VHI/VCI/TCI d'une zone (<code>zone_type</code> , <code>zone_id</code> , <code>years</code> , <code>index_type</code>)
GET	api/champs/geojson/		champs de l'utilisateur
POST	api/champs/create/		créer un champ (<code>nom</code> , <code>date_semi</code> , <code>geom</code> , <code>proprietaire</code> , <code>type_culture</code>)
POST	api/field-ndvi/		historique d'un indice (<code>champ_id</code> , <code>index_type</code>)
POST	api/field-indices-comparison/		plusieurs indices (<code>champ_id</code> , <code>indices[]</code>)
POST	api/field-climate/		pluie CHIRPS + température ERA5 (<code>champ_id</code> , <code>start_date</code>)
POST	api/field-climate-risk/		risque climatique (<code>champ_id</code> , <code>reference_date</code>)
POST	api/field-realtime-status/		situation et prévision à 15 j : CHIRPS-GEFS, tendance CHIRPS, prévision Open-Meteo, SPI prévisionnel, alerte de synthèse
POST	api/field-chirps-gefs/		prévision CHIRPS-GEFS (<code>forecast_days</code>)
POST	api/field-forecast-spi/		SPI prévisionnel (<code>years_history</code>)
POST	api/field-drought-indices/		VHI/VCI/TCI d'un champ

Méthode	Route		Rôle
GET	<code>api/alerts/?status=&champ_id=&limit=</code>		alertes de l'utilisateur
GET	<code>api/champ-risk/<id>/</code>		dernière évaluation de risque
POST	<code>api/evaluate-champ/</code>		évaluer un champ maintenant (<code>champ_id</code> , <code>collect</code> , <code>dry_run</code>)
GET	<code>api/crop-calendar/</code>		calendrier cultural
POST	<code>api/crop-calendar/add/</code> , <code>update/<pk>/</code>	 aucun	créer / modifier
POST, DELETE	<code>api/crop-calendar/delete/<pk>/</code>	 aucun	supprimer
POST	<code>api/crop-calendar/simulate/</code>		stade de croissance (<code>calendar_id</code> , <code>day</code>)

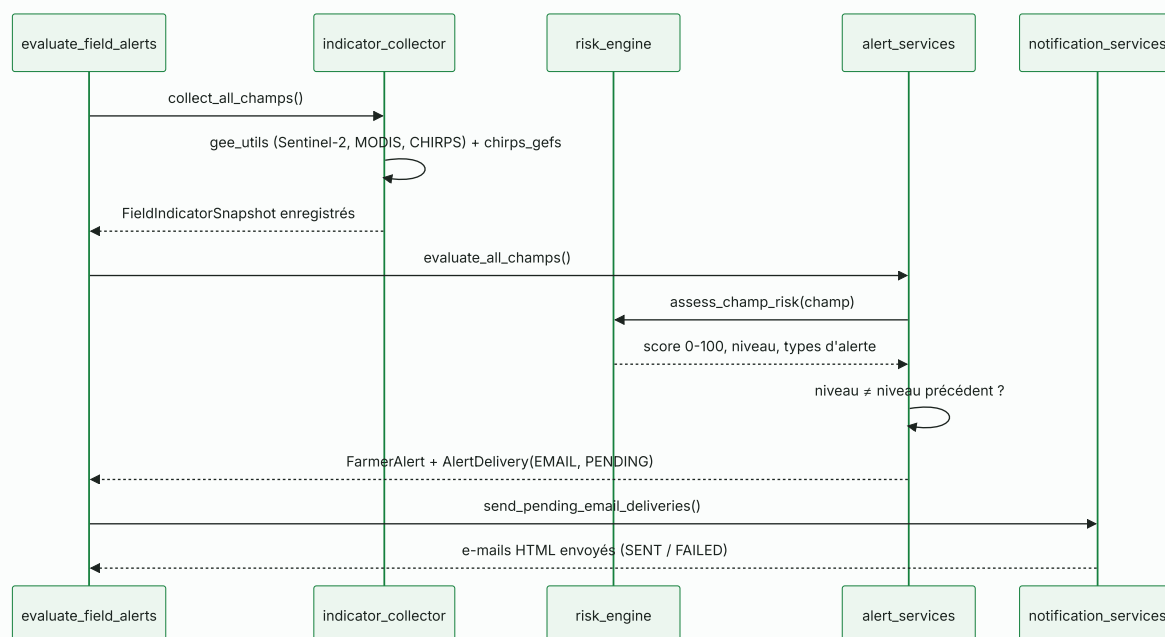
5.4 Cartothèque — `/api/cartotheque/` (router DRF)

Route	Accès	Détails
<code>domaines/</code>	lecture seule, public	domaines actifs ; <code>?search=</code> , <code>?ordering=ordre</code>
<code>cartes/</code>	lecture publique ; écriture selon les permissions Django (<code>add/change/delete_cartetheematique</code>)	<code>?statut=publie</code> (défaut) <code>brouillon</code> <code>tous</code> , <code>?domaine=<id></code> , <code>?domaine_slug=</code> , <code>?search=</code> , <code>?ordering=</code>

6. Système d'alertes agricoles

6.1 Chaîne de traitement

`python manage.py evaluate_field_alerts` enchaîne quatre modules :



- Collecte** (`indicator_collector.py`) : pour chaque champ, appelle `gee_utils` et `chirps_gefs` . Chaque indice est protégé : si l'un échoue, les autres sont quand même enregistrés.
- Score** (`risk_engine.py`) : 5 sous-scores de 0 à 100, pondérés :

Dimension	Poids	Indicateurs
Végétation	0,30	NDVI, EVI, MSAVI (baisse + valeur absolue)
Humidité	0,20	NDWI, corroboré par EVI / MSAVI
Sécheresse	0,25	SPI_30, SPI_90, SPI_FORECAST
Précipitations	0,15	RAINFALL_24H, RAINFALL_72H (excès ou déficit)
Prévision	0,10	SPI_FORECAST, pluie prévue

Les poids sont dans la constante `WEIGHTS` (leur somme doit valoir 1). Niveaux : 0-25 `NORMAL` , 26-50 `VIGILANCE` , 51-75 `ALERTE` , 76-100 `CRITIQUE` . 3. **Décision** (`alert_services.py`) : une alerte n'est créée **que si le niveau change** par rapport au dernier `RiskAssessment` (1re détection, aggravation, amélioration, fin d'alerte). Types : `STRESS_VEGETATIF` , `STRESS_HYDRIQUE` , `SECHERESSE` , `EXCES_PLUIE` , `GENERIC` . Un `AlertDelivery` EMAIL n'est créé que si le profil a `recevoir_alertes=True` . 4. **Envoi** (`notification_services.py`) : e-mail HTML (niveau, score, tableau des indicateurs contributeurs, recommandation).

6.2 Indicateurs

Code	Source	Sens
NDVI , EVI , MSAVI	Sentinel-2 (masque nuages), via GEE	vigueur de la végétation
NDWI	Sentinel-2	eau dans la végétation

Code	Source	Sens
VCI , TCI , VHI	MODIS NDVI (MOD13A2) + température de surface (MOD11A2), via GEE	sécheresse (végétation, température, combiné) ; la collecte quotidienne enregistre le VHI
NCWSI	MODIS NDVI / température de surface, via GEE	stress hydrique normalisé
SPI_30 , SPI_90	CHIRPS, climatologie sur 10 ans	anomalie de pluie observée
SPI_FORECAST	CHIRPS-GEFS	anomalie de pluie prévue sur 15 jours
RAINFALL_24H , RAINFALL_72H	GPM IMERG (NASA), repli sur CHIRPS-GEFS	pluie récente

`weather_forecast.py` (Open-Meteo, sans clé) détecte en plus les **épisodes secs** prévus (7 / 10 / 14 jours consécutifs < 1 mm) et les **cumuls d'inondation** sur 3 jours (60 / 100 / 150 mm).

6.3 Earth Engine

`gee_utils.initialize_ee()` lit la clé de compte de service `ee-koutoumbogajules-c99000ca569e.json` dans le répertoire courant, puis appelle `ee.Initialize(credentials)`. Principales fonctions : `get_monthly_ndvi_series`, `get_8_day_ndvi_series`, `get_field_ndvi_history`, `get_field_indices_history`, `get_chirps_precipitation_series`, `get_era5_temperature_series`, `compute_climate_risk`, `compute_observed_spi_windows`, `get_realtime_precipitation`, `get_drought_index_map_url`.

7. Données et sources

7.1 Couches du géoportail

Les données de terrain d'origine sont dans la base de l'auteur. Sans elles, `python manage.py import_blitta2` reconstruit les couches :

Étape	Source	Détail
Cantons	OCHA COD-AB Togo, niveau ADM3	P-codes <code>TG010101</code> Agbandi, <code>TG010110</code> Langabou, <code>TG010109</code> Koffiti, <code>TG010115</code> Tcharè-Baou
Commune	Union des 4 cantons	Composition : décret publié au JO du 08/01/2018, en application de la loi n°2017-008
Infrastructures	OpenStreetMap via Overpass	Emprise des 4 cantons, puis découpage au contour de la commune
Santé	Inventaire OMS/KEMRI (Maina et al., 2019)	Rapprochement à 2,5 km avec les objets OSM (coordonnées arrondies)

Règles de correspondance OSM :

Couche	Critère OSM
Routes	<code>highway=trunk/primary</code> → <i>Route nationale revêtue</i> ; <code>secondary/tertiary/unclassified/track</code> → <i>Piste rurale</i> (sauf réf. <code>N...</code> revêtue)
Lycées / collèges / jardins	<code>amenity=school/kindergarten</code> classé par nom (<code>Lycée</code> , <code>CEG</code> , <code>Collège</code> , <code>Jardin</code> ...) ; les écoles primaires n'ont pas de couche
Santé	<code>amenity=hospital/clinic/doctors/health_post</code> , <code>healthcare=*</code> ; type déduit du nom (USP, CMS, CHP...) ; doublons < 50 m fusionnés
Marchés	<code>amenity=marketplace</code>
Points d'eau	<code>man_made=water_well/borehole</code> (type selon <code>pump=*</code>)
Bornes-fontaines	<code>amenity=drinking_water/water_point</code> , <code>man_made=water_tap</code>
Châteaux d'eau	<code>man_made=water_tower</code> , <code>storage_tank</code> (+ <code>content=water</code>)
Terrains	<code>leisure=pitch/stadium</code> (polygones)
Coopératives / intrants	<code>office=cooperative</code> , <code>shop=agrarian/farm</code>

Canton : intersection spatiale. Localité : `place=*` le plus proche. Valeur inconnue : `Non renseigné` .

Licences : les données OSM sont sous licence **ODbL**. Toute carte publiée doit mentionner « © contributeurs OpenStreetMap ».

7.2 Cartothèque

`cartotheque/fixtures/initial_domaines.json` : 14 domaines thématiques.

8. Front-end

Pages servies par Django

- `geoportail/templates/index.html` + `geoportail/static/js/script.js` : carte Leaflet centrée sur [8.08, 1.12] (zoom 14), fonds OSM / Esri / Google, couches GeoJSON chargées en parallèle (`initMap`), icônes dans `static/icone/` , outils de dessin et de mesure (Leaflet.draw + Turf), filtres et statistiques par canton. `scripts.js` est une ancienne version reposant sur un GeoServer local (`localhost:8089`) ; seul `script.js` est chargé par le gabarit.
- `agriculture/templates/agriculture/index.html` + `static/js/main.js` : carte simple (centrée sur Aného, couches GeoServer locales).

Fond OSM : Django envoie `Referrer-Policy: same-origin` par défaut. Les couches de tuiles OSM déclarent donc `referrerPolicy: 'strict-origin-when-cross-origin'` ; sans cette option, OSM répond 403 « Access blocked ».

Frontend externe

La configuration CORS (`localhost:3001`), l'authentification par jeton et les icônes « Lucide » de la cartothèque indiquent une application frontend séparée (probablement React), **absente de ce dépôt**, qui consomme les API `/auth/` , `/agriculture/api/` et `/api/cartotheque/` .

9. Commandes de gestion

Commande	Rôle
<code>python manage.py migrate</code>	crée / met à jour les tables gérées
<code>python manage.py loaddata cartotheque/fixtures/initial_domaines.json</code>	domaines de la cartothèque
<code>python manage.py import_blitta2 [--cache-dir D] [--refresh] [--sans-oms]</code>	reconstruit les couches du géoportail (écrase leur contenu)
<code>python manage.py evaluate_field_alerts [--champ-id N] [-- dry-run] [--no-collect] [--no-email]</code>	chaîne d'alertes complète
<code>python manage.py createsuperuser</code>	compte administrateur
<code>python manage.py collectstatic</code>	copie les fichiers statiques dans <code>staticfiles/</code> (production)

10. Sécurité

- **Secrets** : uniquement dans `.env` ou les variables d'environnement. `.env` , les clés `ee-*.json` , `db.sqlite3` , `*.log` et `staticfiles/` sont ignorés par Git.
- **Historique Git** : d'anciens commits contiennent des mots de passe d'application Gmail et une `SECRET_KEY` . Ils doivent être **révoqués** (voir la PR n°1). Les retirer du code ne les efface pas de l'historique.
- **Points à durcir** avant une mise en ligne (détails dans [4-AMELIORER.md](#)) :
 - endpoints `crop-calendar` add / update / delete sans authentification et exemptés de CSRF ;
 - `CORS_ALLOW_ALL_ORIGINS = True` ;
 - `DEBUG` doit rester à `False` en production (c'est la valeur par défaut) ;
 - le mot de passe `postgres` par défaut (`1234`) doit être changé.

4. Améliorer le projet

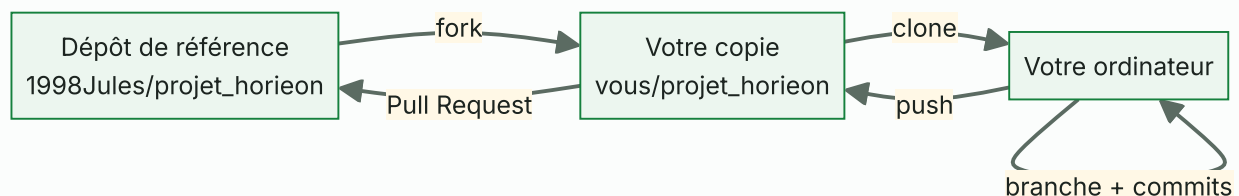
Pour toute personne qui veut **faire évoluer** E-commune : développeurs, étudiants, bénévoles.
Ceux qui ne codent pas peuvent aussi aider : voir [Contribuer sans coder](#).

Sommaire

1. Proposer une modification (fork et Pull Request)
2. Recettes pas à pas
3. Limites connues
4. Feuille de route proposée
5. Contribuer sans coder

1. Proposer une modification (fork et Pull Request)

Le dépôt de référence est https://github.com/1998Jules/projet_horieon. Si vous n'en êtes pas collaborateur, passez par un **fork** (votre copie personnelle) et une **Pull Request** (PR : demande d'intégration).



Étapes

```
# 1. Une seule fois : forker sur GitHub (bouton « Fork »), ou avec GitHub CLI :
gh repo fork 1998Jules/projet_horieon --clone
cd projet_horieon

# 2. Pour chaque amélioration : partir d'une version à jour
git checkout main
git pull origin main
git checkout -b corrige-selection-canton      # nom de branche explicite
```

```
# 3. Modifier, puis vérifier (voir la check-list ci-dessous)

# 4. Enregistrer
git add fichier1 fichier2
git commit -m "Corrige la liste « Choisir canton » du géoportail"

# 5. Publier et proposer
git push -u origin corrige-selection-canton    # origin = votre fork
gh pr create --repo 1998Jules/projet_horieon --fill
```

Check-list avant d'ouvrir une PR

- ☐ `python manage.py check` ne signale aucune erreur.
- ☐ `python manage.py makemigrations --check --dry-run` : si vous avez modifié un modèle, la migration est incluse dans la PR.
- ☐ Les pages touchées ont été testées dans le navigateur, **console JavaScript comprise** (F12 > Console).
- ☐ **Aucun secret** dans les fichiers : mots de passe, `SECRET_KEY`, clés `ee-*.json`, fichier `.env`.
- ☐ Les fichiers sont en **UTF-8**. Sous Windows, `pip freeze > requirements.txt` dans PowerShell produit de l'UTF-16 que `pip` ne sait pas relire : préférez `pip freeze | Out-File -Encoding utf8 requirements.txt`, ou éditez le fichier à la main.
- ☐ Pas de fichiers générés : `__pycache__` / , `staticfiles` / , `db.sqlite3` , `*.log` (déjà dans `.gitignore`).
- ☐ La description de la PR explique **quoi**, **pourquoi** et **comment c'est testé**.
- ☐ La documentation (`docs` /) est mise à jour si le comportement change.

Conventions

- Code, commentaires et messages de commit **en français** (comme le reste du projet), noms de variables en anglais ou en français selon le fichier modifié.
- Un commit = une idée. Message à l'infinitif ou au présent : « Ajoute... », « Corrige... ».
- Configuration : jamais de valeur sensible dans le code ; ajoutez une variable dans `.env.exemple` et lisez-la avec `os.environ.get(...)` dans `horison/settings.py`.

2. Recettes pas à pas

2.1 Ajouter une couche au géoportail

Exemple : une couche « Mairies ».

1. **Modèle** (`geoportail/models.py`) :

```
class mairie(models.Model):
    canton_nom = models.CharField(max_length=100)
    nom = models.CharField(max_length=100)
    geom = models.PointField(srid=4326)

    class Meta:
        db_table = "mairies"
        managed = False # comme les autres couches ; ou True pour qu'une migration crée
                          la table
```

Avec `managed = False`, créez la table vous-même (import QGIS, `shp2pgsql`), ou ajoutez son remplissage à `import_blitta2`, qui crée automatiquement les tables manquantes de l'application `geoportail`.

2. **Vue** (`geoportail/views.py`), sur le modèle des autres :

```
def mairie_geojson(request):
    qs = mairie.objects.exclude(geom__isnull=True)
    geojson = serialize('geojson', qs, geometry_field='geom', fields=('canton_nom',
    'nom'))
    return HttpResponse(geojson, content_type='application/json')
```

3. **Route** (`geoportail/urls.py`) : `path('geojson/mairie/', views.mairie_geojson,`
`name='mairie_geojson')`,

4. **Carte** (`geoportail/static/js/script.js`) :

- une icône dans `geoportail/static/icone/mairie.png` et sa déclaration `L.icon(...)` à côté de `infraIcons` ;
- la couche : `geoLayers["Mairies"] = L.geoJSON(null, { pointToLayer: ..., onEachFeature:`
`... });`
- son chargement dans `initMap()` : `loadGeoJSON("/geoportail/geojson/mairie/",`
`geoLayers["Mairies"])`,
- son rattachement au bon **thème** dans le panneau des couches.

5. **Recherche** (facultatif) : ajouter la couche à `SEARCHABLE_LAYERS` dans `views.py`.

6. **Tester** : `/geoportail/geojson/mairie/` renvoie une `FeatureCollection`, puis la couche apparaît sur la carte.

2.2 Mettre à jour les données depuis OpenStreetMap

Après des ajouts dans OpenStreetMap :

```
python manage.py import_blitta2 --refresh
```

`--refresh` retélécharge les sources au lieu d'utiliser le cache (`%TEMP%\horieon_import` par défaut, modifiable avec `--cache-dir`).

2.3 Adapter l'import à une autre commune

Dans `geoportail/management/commands/import_blitta2.py` :

1. remplacez le dictionnaire `CANTONS` par les **P-codes COD-AB** des cantons de la commune (liste dans le fichier `tgo_admin3.geojson` du jeu `cod-ab-tgo`) et leurs noms officiels ;
2. adaptez le nom et les codes de la commune dans `load_boundaries()` ;
3. changez le centre de la carte dans `geoportail/static/js/script.js` (`setView([lat, lon], zoom)`).

La composition des communes est fixée par le décret publié au *Journal officiel* du 8 janvier 2018 (application de la loi n°2017-008).

2.4 Ajuster le score de risque agricole

- **Pondérations** : constante `WEIGHTS` dans `agriculture/risk_engine.py` (la somme doit rester égale à 1, une assertion le vérifie).
- **Seuils d'une dimension** : fonctions `_score_vegetation` , `_score_humidity` , `_score_drought` , `_score_rainfall` , `_score_forecast` .
- **Niveaux** : `_level_from_score` .
- **Messages et recommandations** : `ALERT_TEMPLATES` dans `agriculture/alert_services.py` .
- **Règles d'alerte** : modèle `AlertRule` , modifiable depuis `/admin/` (règles par défaut : migration `0008_default_alert_rules.py`).

Testez sans rien envoyer : `python manage.py evaluate_field_alerts --champ-id <id> --dry-run` .

2.5 Ajouter un indicateur agricole

1. Ajoutez son code dans `FieldIndicatorSnapshot.INDEX_CHOICES` (`agriculture/models.py`), puis `python manage.py makemigrations agriculture` .
2. Calculez-le dans `gee_utils.py` (ou un nouveau module).
3. Collectez-le dans `indicator_collector.py` .
4. Utilisez-le dans une dimension de `risk_engine.py` .

2.6 Ajouter l'envoi par WhatsApp

Le modèle est prêt (`AlertDelivery.CHANNELS` contient `WHATSAPP` , `UserProfile.whatsapp` stocke le numéro). Il reste à :

1. créer les `AlertDelivery(channel="WHATSAPP")` dans `alert_services.py` ;
2. écrire `send_pending_whatsapp_deliveries()` (API WhatsApp Business, ou un fournisseur SMS comme Twilio) dans `notification_services.py` ;
3. l'appeler depuis `evaluate_field_alerts` .

3. Limites connues

Classées par gravité. Les numéros de ligne correspondent à la version de ce document.

Sécurité

#	Problème	Où	Piste
S1	Des mots de passe Gmail et une <code>SECRET_KEY</code> sont dans l'historique Git	anciens commits	Les révoquer côté Google et en production. Réécrire l'historique (<code>git filter-repo</code>) ne suffit pas si le dépôt a été cloné
S2	<code>crop-calendar/add</code> , <code>update</code> , <code>delete</code> : aucune authentification , CSRF désactivé	<code>agriculture/views.py</code> (<code>add_crop_calendar</code> , <code>update_crop_calendar</code> , <code>delete_crop_calendar</code>)	Ajouter <code>@require_api_user</code> + vérification <code>is_staff</code>
S3	<code>CORS_ALLOW_ALL_ORIGINS = True</code>	<code>horison/settings.py:110</code>	En production : <code>CORS_ALLOWED_ORIGINS</code> lu depuis <code>.env</code>
S4	Mot de passe base par défaut <code>1234</code>	<code>settings.py</code> , exemples	Imposer <code>DB_PASSWORD</code> en production

Fonctionnement

#	Problème	Où	Piste
F1	Pas de page d'accueil (404 sur <code>/</code>)	<code>horison/urls.py</code>	<code>path('', RedirectView.as_view(url='/geoportail/'))</code>
F2	La liste « Choisir canton » reste vide : le code lit <code>properties.cant</code> , l'API renvoie <code>canton</code>	<code>geoportail/static/js/script.js:398-401</code>	Lire <code>feature.properties.canton</code> ; faire zoomer la carte sur le canton choisi
F3	Pas de couche « quartiers » (<code>allQuartiers</code> jamais rempli)	<code>script.js</code>	Ajouter une couche quartiers (voir 2.1)
F4	Vues <code>terrain_geojson</code> , <code>cooperative_geojson</code> , <code>magazin_geojson</code> non routées	<code>geoportail/urls.py</code>	Ajouter les <code>path(...)</code> et charger les couches dans <code>initMap()</code>
F5	<code>api/champs/</code> pointe vers <code>ndvi_timeseries</code> (copier-coller)	<code>agriculture/urls.py:22</code>	Vérifier l'usage côté frontend, puis corriger ou supprimer
F6	Tables <code>region</code> , <code>couche_prefecture_utm</code> , <code>communes_togo_utm</code> absentes : erreur 500 sur <code>api/regions/</code> , <code>prefectures/</code> , <code>communes/</code>	<code>agriculture/models.py</code>	Les remplir depuis COD-AB (ADM1, ADM2, communes) avec une commande d'import
F7	Carte agriculture centrée sur Aného, couches GeoServer <code>localhost:8089</code>	<code>agriculture/static/js/main.js</code>	Centrer sur Blitta 2, servir les couches depuis Django

#	Problème	Où	Piste
F8	Clé Earth Engine : nom de fichier codé en dur, chemin relatif au dossier de lancement	agriculture/ge e_utils.py:14	Variable GEE_SERVICE_ACCOUNT_KEY (chemin absolu) + GEE_PROJECT dans .env
F9	run_alerts.bat et install_alerts_task.ps1 contiennent des chemins D:\Horison\...	racine	Chemins relatifs (%~dp0) et venv .venv
F10	Réponses GeoJSON sans charset	geoportail/vie ws.py	content_type='application/json'; charset=utf-8'
F11	Langue et fuseau en-us / UTC	settings.py:20 4	fr-fr et Africa/Lome (même heure qu'UTC)
F12	X_FRAME_OPTIONS = "ALLOW-FROM ..." obsolète	settings.py:21 9	En-tête Content-Security- Policy: frame-ancestors
F13	Alertes seulement par e-mail	notification_s ervices.py	Voir 2.6

Qualité du code

#	Problème	Piste
Q1	Aucun test automatisé (tests.py vides)	Commencer par les vues GeoJSON, risk_engine (fonctions pures, faciles à tester) et import_blitta2
Q2	Fichiers en double ou inutilisés à la racine : urls.py , wsgi.py , asgi.py , __init__.py , model.py , commune.geojson , et geoportail/static/js/scripts.js	Vérifier qu'aucun déploiement ne les utilise, puis les supprimer
Q3	Fonctions redéfinies dans le même fichier (ex. geojson_to_ee_geometry deux fois dans gee_utils.py , imports répétés dans views.py)	Nettoyage
Q4	Noms de tables et de modèles hétérogènes (chatea , formation_s , classes en minuscules)	Harmoniser lors d'une future migration de données
Q5	Pas d'intégration continue	GitHub Actions : pip install , manage.py check , tests, avec un service PostGIS

4. Feuille de route proposée

Priorité	Objectif	Tâches	Effort
P0 — urgent	Sécurité	S1 (révocation des secrets), S2, S3	½ journée
P1	Application utilisable partout	F1, F2, F6, F8, F9	2 à 3 jours

Priorité	Objectif	Tâches	Effort
P1	Fiabilité	Q1 (premiers tests), Q5 (CI)	2 jours
P2	Données	Campagne de collecte terrain (jours de marché, points d'eau, châteaux d'eau, coopératives) ; contribution à OpenStreetMap ; couche quartiers (F3) ; couches F4	variable
P2	Agriculture	F7 ; WhatsApp / SMS (F13) ; tableau de bord des alertes dans l'admin	1 à 2 semaines
P3	Confort	Page d'accueil, interface mobile, export PDF des cartes de la cartothèque, traduction en langues locales (éwé, kabiyè...)	variable

5. Contribuer sans coder

- **Compléter OpenStreetMap** (<https://www.openstreetmap.org>, compte gratuit) : ajouter un marché, un forage, une école, le nom d'une piste. Ces ajouts sont repris au prochain `import_blitta2 --refresh`. Les applications mobiles **StreetComplete** ou **Every Door** permettent de le faire sur le terrain.
- **Signaler une erreur** : ouvrir une *issue* sur https://github.com/1998Jules/projet_horieon/issues en décrivant ce qui ne va pas (lieu, capture d'écran).
- **Collecter des données de terrain** (formulaire papier ou application comme KoboToolbox / ODK) : nom, position GPS, canton, localité, et selon le type : jour de marché, type de pompe, statut public / privé, date d'ouverture...
- **Tester** une nouvelle version et faire part de vos retours.
- **Améliorer cette documentation** : une explication pas claire est un défaut comme un autre.